

NOIP 2026 模拟赛题解

2026 年 9 月 22 日

目录

1	bean	1
2	find	2
3	tree	3
4	race	4

A 吃豆人 (bean)

题目来源：洛谷 P14541

考虑吃豆人的行动逻辑，可以发现以下性质：所有吃豆人位置的相对顺序不会改变。所以，把吃豆人按它们初始的相对顺序来排列，那么一个黄豆会让一段前缀的吃豆人位置加上某个值，并让一个后缀的吃豆人位置减去某个值。

怎么知道一个黄豆，吃它的吃豆人是谁以及要花多久吃到它呢？就相当于对它的位置求前驱后继，并求同在某个位置的所有吃豆人，最小的编号。

也就是说，若用数据结构维护所有吃豆人，则该数据结构需支持位置的区间加，求前驱后继，以及区间求最小编号；线段树即可。

时间复杂度 $O((n + m)\log n)$

(PS: 实际上，由于两个吃豆人位置从不同变为相同，只有面对面同时吃到一个黄豆，这一种情况；所以可以用并查集合并所有相同位置的吃豆人，并维护同一位置吃豆人的最小编号，遇见以上状况做一次合并即可。这样就只剩下区间位置加减，以及求前驱后继两种操作了。我们把区间加减差分成单点操作，求前驱后继，就变成了求前缀和第一个 $\geq x$ 的位置之类的。这个可以从高位到低位，在树状数组上二分完成。以上做法时间复杂度也是 $O((n + m)\log n)$ ，但可以优化常数。提及的原因是验题人认为搬题人，把这题数据范围加强到 10^6 ，且开 $1s$ ，必有高论。但卡掉线段树的做法，实在是没必要，所以决定不卡，只是提一下树状数组的做法)

B 路径寻找 (find)

题目来源：洛谷 P17035

直接在原图上进行 DFS。

一开始将所有点都归入集合 A 。每次 DFS 进入一个点，就将其移出 A ；DFS 退出一个点，就将其加入集合 B 。也就是说，还没有 DFS 到的点属于 A ，已经 DFS 结束的点属于 B ，当前还在 DFS 栈中的点构成路径 P 。

由 DFS 的性质，集合 A 与集合 B 之间显然没有边相连。

同时， $|B| - |A|$ 一开始为负，最终非负，且每次只变化 1，因此过程中必然存在一个时刻满足

$$|A| = |B|.$$

找到这个时刻，即得到一种合法构造。

时间复杂度 $O(n + m)$ 。

C 补林为树 (tree)

题目来源：洛谷 P7026

若原森林只有一个连通块，就是经典的树上最大匹配，直接树形 DP 即可。

现在考虑有多个连通块。先在每个连通块内做树形 DP。对于一个以 u 为根的连通块，设 $f_{u,0/1}$ 表示根 u 不匹配/匹配时能得到的最大匹配大小。

显然有

$$f_{u,1} = f_{u,0}$$

或

$$f_{u,1} = f_{u,0} + 1.$$

若 $f_{u,0} = f_{u,1}$ ，我们将这个连通块称为盈树；若 $f_{u,1} = f_{u,0} + 1$ ，则称为亏树。

直观上，对于盈树，我们应当将根空出来不匹配；对于亏树，则应在块内匹配根。

对于盈树，若将根空出来，则它还有可能与更上面的点匹配并额外产生收益；若已经在块内匹配根，则这个连通块对更上面没有价值。

对于亏树，若将根空出来，最好的情况也只是与更上面的点匹配，得到 1 的收益，同时还会消耗更上面的一个未匹配点；而若直接在块内匹配根，同样有 1 的收益，且不会影响更上面。因此亏树一定选择在块内匹配根。

于是，对于盈树，我们希望将其挂在一个未匹配点上，使它的根能够与该点匹配并带来额外收益；对于亏树则可以随意连接。

因此可以先将所有亏树任意挂在 1 所在的连通块上，然后处理所有盈树。

我们自然希望在挂每棵盈树时，当前已经合并的部分仍然存在未匹配点，因为这样答案可以额外 +1。因此将所有盈树按照其未匹配点数从大到小排序，优先处理未匹配点更多的盈树显然更优。

一个连通块在最大匹配下的未匹配点数为

$$\text{总点数} - 2 \times \text{最大匹配}.$$

设当前已经合并部分有 x 个未匹配点，新加入的盈树有 y 个未匹配点。若 $x > 0$ ，则将新树的根挂在一个未匹配点上，并将这条边加入匹配，此时答案额外加 1，未匹配点数变为

$$x + y - 2.$$

否则只能直接连接，未匹配点数变为

$$x + y.$$

按上述过程模拟即可。时间复杂度 $O(n \log n)$ 。

D 双向比赛 (race)

题目来源: Codeforces 671E

考虑如何用尽可能少的额外燃料使区间 $[l, r]$ 可行。

记

$$sg_i = \sum_{j=1}^i g_j, \quad sw_i = \sum_{j=1}^i w_j.$$

先考虑从 l 走到 r 。到达 x 时所剩燃料为

$$sg_{x-1} - sg_{l-1} - sw_{x-1} + sw_{l-1}.$$

若到达 x 时所剩燃料首次为负, 即

$$sg_{x-1} - sw_{x-1} < sg_{l-1} - sw_{l-1},$$

则必须给 $x-1$ 或其之前的加油站补充额外燃料。

容易发现, 因为之后还需要反向行驶, 所以此时将额外燃料补充在 $x-1$ 显然最优。

记

$$q_x = sg_{x-1} - sw_{x-1}.$$

令 $next_l$ 表示第一个满足 $q_x < q_l$ 的位置, 则需要在 $next_l - 1$ 处补充

$$val_l = q_l - q_{next_l}.$$

单位燃料。

补充后, 相当于需要从 $next_l$ 走到 r , 与原问题完全相同。因此继续沿 $next$ 跳跃, 直到 $next > r$ 为止。

补充完以上所有燃料后, 就一定能够从 l 走到 r , 但未必能够从 r 走回 l 。

设 g'_i 表示经过从 l 往右走的补充后, 第 i 个加油站的燃料量。若还需要补充 t 单位燃料才能从 r 走回 l , 则剩余的燃料全部补充在 r 显然最优。

于是需要满足

$$t + g_r - w_{r-1} \geq 0,$$

$$t + g_r + g'_{r-1} - w_{r-1} - w_{r-2} \geq 0,$$

以及一般地

$$t + g_r + sg'_{r-1} - sg'_x - sw_{r-1} + sw_{x-1} \geq 0.$$

因此, 只需对区间内所有 x 求

$$sg'_x - sw_{x-1}$$

的最大值，即可求出返程所需补充的燃料。

现在梳理整个过程。第一步比较难直接维护，因为需要对

$$nxt_l - 1, \quad nxt_{nxt_l} - 1, \quad \dots$$

这些位置的 g 进行修改，得到 g' ，有些类似链上修改，但修改位置并不是连续的一条链，而且每次增加的值也不同，看起来并不好直接处理。

但我们并不是要对若干组 $[l, r]$ 求最少的额外燃料消耗量，而是要找到最大的额外燃料消耗量不超过 k 的区间。正常的做法是扫描线，比如枚举左端点 l ，并对每个 l 寻找最大的合法 r 。

因此可以调整枚举 l 的顺序。

将 l 连向 nxt_l ，得到一棵树，在这棵树上 DFS 枚举 l 。此时进入或退出 l 对应的子树时，只需要对 $nxt_l - 1$ 位置的 g' 做一次单点修改，而对应到前缀和 sg' 上就只是一次后缀加。

接下来要求的是额外燃料消耗量不超过 k 的最大 r 。

若对 l ，设从 l 走到 r 需要的额外燃料为 c_r ，那总消耗量即为

$$c_r + \max\{0, \max_{l \leq x < r} \{sg'_x - sw_{x-1}\} - g_r - sg'_{r-1} + sw_{r-1}\}$$

。由于显然有， $sg'_{r-1} - sg_{r-1} = c_r$ ，故总消耗量即为

$$\max\{c_r, \max_{l \leq x < r} \{sg'_x - sw_{x-1}\} - sg_r + sw_{r-1}\}$$

。要求其 $\leq k$ ，即要求 $c_r \leq k$ ，且

$$\max_{l \leq x < r} \{sg'_x - sw_{x-1}\} - sg_r + sw_{r-1} \leq k$$

。

前者由于 c_r 单调不降，且每次从 nxt_l 移动到 l ，或是从 l 移动到 nxt_l ， c_r 对应地只会做后缀加减。故可用线段树维护所有 c_r ，在线段树上二分找到最后一个 $\leq k$ 的位置 R ，限定 $r \leq R$ 即可。注意，若 $R = l$ ，则直接取 $r = l$ ；以下假设 $R > l$ 。

后者可设 $sg'_x - sw_{x-1} = A_x$ ， $-sg_r + sw_{r-1} = B_r$ ，那么就是要求

$$\max_{l \leq x < r} \{A_x\} + B_r \leq k$$

。由于在 R 之前，有 $A_x + B_x = c_{x+1} \leq k$ 。故考虑在 $[l, R-1]$ 中，找到 A_x 最后一个最大值所在的位置 x ，那么由于 $A_x + B_x \leq k$ ，故 x 一定可行。其中对 A_x ，要支持的就是后缀加，以及找区间最后一个最大值的位置。线段树维护值和位置的二元组，区间加区间查询，打 lazy tag 即可。

那么就变为了，在 $[x, R]$ 中，找最后一个使得 $A_x + B_r \leq k$ 的位置。此时 A_x 为定值，移项即 $B_r \leq k - A_x$ 。线段树维护区间 B_r 最小值，在线段树上二分即可。

时间复杂度 $O(n \log n)$ 。